Relatório do trabalho final
Protocolo de transporte confiável sobre UDP

Integrantes: Lorenzo Schavinski Eulalio de Souza e Wesley Pereira da Silva
Data: 28/06/2026

Parte 1 — Stop-and-Wait

1. Visão geral da implementação

O SRTP roda sobre UDP e adiciona confiabilidade na aplicação. Cada pacote tem um cabeçalho de 9 bytes big-endian (SYN, FIN, SEQ de 14 bits, ACKflag, NACK, ACK de 14 bits, Length de 8 bits e CRC32 de 32 bits).

O Length codifica a posição no fluxo: 255 = payload cheio e ainda vem mais, valor < 255 = último pacote, 0 = arquivo múltiplo exato do payload e o último bloco cheio já foi enviado. O CRC32 é calculado com zlib.crc32 sobre o cabeçalho (com o campo CRC zerado) mais o payload, protegendo cabeçalho e dados.

A conexão abre por handshake de 3 vias (SYN → SYN+ACK → ACK, cada lado anuncia sua janela e a efetiva é o mínimo entre as duas) e fecha por FIN/FIN+ACK. O timeout é fixo em 100 ms, e no stop-and-wait a janela é 1. O serviço usa duas portas adjacentes (P e P+1), uma por sentido.

2. Throughput teórico vs medido (cenário L0/P0)

Pra comparar com o medido usamos a fórmula de stop-and-wait do Kurose (G_sw = L_pay/(RTT+t_trans)). O payload é 255 B (2040 bits) e o pacote no fio 264 B (2112 bits). No cabo Gigabit o tempo de transmissão dá ~2,1 µs, desprezível perto do RTT base (~0,72 ms), então a vazão fica limitada pelo RTT. Com isso o teórico em L0 dá ~2833 kbit/s, e a gente mediu 2847,3 kbit/s. Para a perda usei G_sw·(1-p), mas é um teto otimista porque não conta o tempo parado esperando o timeout.

3. Cenários de latência L0–L3

Capturas: saw_L0.pcapng, saw_L1.pcapng, saw_L2.pcapng, saw_L3.pcapng

Cenário	**Latência (ms)**	**RTT≈ (ms)**	**Throughput medido (kbit/s)**	**Nº retransmissões**	**Teórico (kbit/s)**
L0	0	0,72	2847,3	0	2833
L1	50	100,7	12,6	147	20,3

L2	100	200,7	9,2	390	10,2
L3	150	300,7	6,3	504	6,8

O RTT é mais ou menos 2× a latência por sentido. Em L1 (~100 ms) ele já encosta no timeout de 100 ms e começam as retransmissões desnecessárias (147), derrubando a vazão. De L1 pra L2 (~200 ms) o RTT passa do timeout: o emissor reenvia antes do ACK voltar (390), e L3 piora (504). Como o timeout é fixo, ele dispara cedo demais e o medido fica abaixo do teórico.

4. Cenários de perda P0–P4

Capturas: saw_P0.pcapng, saw_P1.pcapng, saw_P2.pcapng, saw_P3.pcapng, saw_P4.pcapng

Cenário	**Perda (%)**	**Throughput medido (kbit/s)**	**Nº retransmissões**	**Teórico G_sw·(1-p) (kbit/s)**
P0	0	3176,8	0	2833
P1	1	553,4	6	2805
P2	5	109,0	41	2691
P3	10	67,2	68	2550
P4	25	17,7	263	2125

A perda derruba a vazão porque cada pacote perdido custa um timeout inteiro de 100 ms parado, não só o reenvio. Por isso o medido (553→17 kbit/s) cai muito mais que o teórico G_sw·(1-p) (2805→2125 kbit/s), que assume reenvio na hora. As retransmissões sobem com a taxa de perda (6→263).

5. Comportamento do CRC32

Pacote com CRC32 inválido é descartado em silêncio, sem NACK, e o timeout do emissor retransmite, vira o mesmo caminho de uma perda. Na captura "*saw_crc.pcapng*"dá pra ver o pacote corrompido sem ACK e a retransmissão depois. Sem o CRC, esse pacote seria entregue à aplicação como válido e corromperia o arquivo sem ninguém perceber.

Parte 2 — Go-Back-N e Selective Repeat

1. Visão geral

GBN e SR usam o mesmo cabeçalho, handshake e encerramento da Parte 1, mudando só a política de janela e recepção (janela máxima 255, limite do campo Length).

- **Go-Back-N:** janela de N em voo, receptor só aceita em ordem (fora de ordem é descartado), ACK cumulativo e NACK do esperado. No timeout ou NACK, retransmite todos a partir do primeiro não confirmado.
- **Selective Repeat:** janela de N em voo, receptor bufferiza os fora de ordem e entrega quando a lacuna fecha, ACK individual e NACK do seq faltante. Retransmite só esse pacote.

2. Throughput teórico vs medido (GBN e SR)

Janela deslizante (mesmos L_pay, RTT, t_trans): G_win = min(R·L_pay/L_wire, N·L_pay/(RTT+t_trans)). Com R=1 Gbit/s o termo do enlace (≈966 Mbit/s) não limita; domina o termo da janela, que cresce ~N vezes sobre o stop-and-wait.

Protocolo	**Janela**	**G_win teórico (kbit/s)**	**Medido L0 (kbit/s)**
GBN	4	11333	10184
GBN	16	45333	33987
SR	4	11333	10539
SR	16	45333	41083

O medido fica abaixo do teórico por overhead de protocolo/SO, mas janela maior multiplica o throughput. Em L0 GBN e SR têm o mesmo teórico; divergem sob perda/reordenação.

3. Comparativo sob latência (saw vs GBN vs SR, L0–L3)

Throughput medido (kbit/s). j4 = janela 4, j16 = janela 16.
Capturas: gbn_L0-L3_janela4.pcapng, gbn_L0-L3_janela16.pcapng, sr_L0-L3_janela4.pcapng, sr_L0-L3_janela16.pcapng, saw_L0-L3.pcapng.

Cenário	**saw**	**GBN j4**	**GBN j16**	**SR j4**	**SR j16**	**teór. saw**	**teór. j4**	**teór. j16**
L0	2847,3	10184	33987	10539	41083	2833	11333	45333
L1	12,6	50,0	197,3	50,0	196,9	20,3	81	324
L2	9,2	37,2	145,2	37,4	145,2	10,2	40,7	162,6
L3	6,3	25,0	98,2	25,0	98,2	6,8	27,1	108,5

Com janela N o emissor manda até N pacotes antes de exigir ACK, mantendo o canal cheio durante o RTT. Por isso GBN e SR seguram a vazão em L1–L3 muito melhor que o saw, e janela 16 aguenta latências mais altas que janela 4.

4. Comparativo sob perda (GBN vs SR, P1–P4)

Throughput em kbit/s. *Capturas: gbn_P1-P4_janela4.pcapng, gbn_P1-P4_janela16.pcapng, sr_P1-P4_janela4.pcapng, sr_P1-P4_janela16.pcapng.*

Janela 4:

Cenário	Perda (%)	GBN tput	GBN retr	SR tput	SR retr
P1	1	9384,8	24	2611,4	13
P2	5	7296,6	184	226,7	54
P3	10	6881,0	224	163,7	100
P4	25	292,1	569	44,7	261

Janela 16:

Cenário	Perda (%)	GBN tput	GBN retr	SR tput	SR retr
P1	1	16538,6	432	1151,4	39
P2	5	7071,9	2816	356,2	114
P3	10	6781,2	2061	272,2	171
P4	25	5108,8	3821	98,6	299

Quando perde um pacote, o GBN retransmite o lote a partir do não confirmado; o SR retransmite só o faltante. Daí o GBN gerar muito mais retransmissões, mas com throughput bruto maior. SR é mais eficiente (retransm. perto do nº de perdas), GBN é mais agressivo.

5. Comparativo sob reordenação R0–R2

Capturas: gbn_R0-R2_janela4.pcapng, gbn_R0-R2_janela16.pcapng, sr_R0-R2_janela4.pcapng, sr_R0-R2.janela16.pcapng

Janela 4:

Cenário	Reord. (%)	GBN tput	GBN retr	SR tput	SR retr
R0	0	10138,0	0	10727,1	0
R1	10	9323,1	103	10179,0	9
R2	25	8949,7	232	10114,8	23

Janela 16:

Cenário	Reord. (%)	GBN tput	GBN retr	SR tput	SR retr
R0	0	31704,9	0	39330,5	0
R1	10	8004,3	1936	37024,4	6
R2	25	6135,3	2367	33085,0	17

A reordenação não perde pacote, só entrega fora de ordem. No GBN o pacote fora de ordem é descartado, gerando NACK e retransmissão do lote (trata quase como perda). No SR o pacote fora de ordem é bufferizado; ele manda NACK apenas do faltante, sendo quase imune à reordenação. O SR preserva a vazão e minimiza retransmissões; o GBN tem desempenho degradado.

6. Conclusão comparativa

Juntando o que medimos, dá pra dizer em que situação cada protocolo se sai melhor:

- **Canal limpo e de baixa latência**: os três entregam o arquivo sem problema. Como o stop-and-wait é o mais simples de implementar e entender, a janela deslizante não traz ganho real quando quase não há atraso nem perda.
- **Latência alta, pouca perda**: a janela deslizante vira essencial. O saw colapsa porque espera um RTT inteiro a cada pacote e ainda sofre com o timeout fixo, enquanto GBN e SR mantêm vários pacotes e seguram muito mais vazão. Entre GBN e SR o desempenho é praticamente igual sob latência pura, então compensa o mais simples, o GBN.
- **Perda significativa**: o SR é mais eficiente no uso da rede, porque retransmite só o pacote que faltou, enquanto o GBN reenvia o lote inteiro a partir do não confirmado. Se o que importa é eficiência, SR; se é só empurrar dados rápido sem ligar pro desperdício, GBN.
- **Reordenação significativa**: o SR é disparado a melhor escolha. Como bufferiza o que chega fora de ordem, a reordenação quase não o afeta. Já o GBN trata reordenação quase como perda — descarta o que chegou adiantado e retransmite em lote.

Sobre o tamanho de janela: aumentar a janela multiplica a vazão em rede limpa, mas amplifica o desperdício do GBN quando a rede piora, o SR escala bem nos dois casos porque a retransmissão é sempre pontual e não cresce com N. No fim não existe um "melhor protocolo" absoluto: o stop-and-wait serve pra redes boas e curtas, o GBN é um meio-termo simples que sofre com perda e reordenação, e o SR é o mais robusto e eficiente, ao custo de mais complexidade no receptor.

No fim, conseguimos entender por que protocolos reais como o TCP usam janela deslizante com timeout adaptativo no lugar de um valor fixo, e por que a forma de retransmitir (em lote ou seletiva) muda tudo dependendo da rede.